

# DragonWings Power Availability Profile Forecaster

Final Capstone Report Section B

Shawn Cunningham  
Carnegie Mellon University and ExecOnline Agentic AI Program  
September 7, 2026

## Project summary

DragonWings PAP is a working, read-only agent service that estimates additional solar power available over the next 12 hours. It combines real MySolArk telemetry, deterministic calculations, durable workflow state, retrieved memory, and bounded model roles. The MVP demonstrates integration and inspectable decisions. Its present evidence supports implementation correctness and bounded execution, while forecast accuracy and model quality still require broader evaluation.

## Problem and intended user

An operator of a portable solar and battery system needs to decide whether additional electrical load is reasonable. A current power reading cannot establish future availability because solar generation and existing demand change. A fluent language-model answer is insufficient: the answer requires fresh measurements, consistent units, explicit constraints, and a record of how the calculation was produced.

The intended users are DragonWings operators and engineers reviewing optional-load plans. PAP presents an interval profile, supporting evidence, and guidance or a withholding reason. The operator retains responsibility for equipment decisions. The service has no path for commanding an inverter, battery, generator, or connected load.

## Goal scope and evaluation criteria

The goal is an explainable 12-hour profile of additional power in kW and interval energy in kWh. After an operator requests a run, the workflow autonomously acquires evidence, validates it, calculates availability, selects any needed model interpretation, and publishes or withholds. Outcome evaluation and memory indexing are explicit operator actions.

The smallest working scope uses the latest persisted DW 1.24 MySolArk scrape, synthetic hourly solar factors, constant measured demand, and zero battery-discharge allowance. It provides evaluation guidance, with live equipment capability still unconfigured. Success criteria are correct arithmetic and units, rejection of stale evidence and observed voltage-floor violations, bounded model execution, durable inspection records, and reproducible local browser behavior. Forecast error and the optional third agent are evaluated separately. [R1, R2]

## Final architecture and major components

LangGraph coordinates one typed workflow. PostgreSQL stores canonical telemetry, forecasts, outcomes, provenance, and audit records. The official PostgreSQL checkpointer stores workflow references and control state, allowing execution to resume without treating checkpoint data as a second source of operational facts.

T1 and T2 expose two read-only Model Context Protocol tools for telemetry and weather inputs. T3 validates schemas, units, UTC scrape freshness, and the weather horizon. Deterministic Python implements T4 forecasting, T5 availability arithmetic, and T6 constraints. Available power is nonnegative solar surplus, subject to an equipment cap when configured. Energy integrates each interval once.

The routine route can proceed without model generation. Recorded solar overestimation above a demonstration threshold of 0.25 kW triggers selective search. T9 retrieves qualified memory. LangChain create\_agent nodes generate and critique structured guidance using the same evidence. Python prunes candidates, controls the search, and retains numerical authority. T7 rechecks freshness and constraints before publishing or withholding. T8 compares later observations with the forecast and stores feedback for subsequent runs. [R2]

## Implementation approach

The native command-line service uses Python, FastAPI, LangGraph, LangChain, and PostgreSQL with pgvector. Environment settings select local Ollama, OpenAI, or Claude. Gemma 3 4B supplies the demonstrated local responses; nomic-embed-text supplies 768-dimensional memory vectors. Cosine retrieval considers five candidates and admits at most three after source, model-version, expiry, relevance, and duplicate checks. Similarity scores rank relevance, not probability.

The HTML, CSS, and JavaScript console follows the conversation-and-inspector idea in Agent-with-Subagent, without Gradio. Six tabs expose context, memory, tools, subagents, trace, and health. A linked run restores its evidence and records; readable summaries precede raw details. Small PRs built the vertical slice with terse teaching lessons. Docker and Deep Agents are excluded. [R1, R2, R7, R8]

## Design evolution across checkpoints

Each checkpoint added a capability and clarified its authority. The final implementation narrows several early design ambitions so the full workflow can be demonstrated locally. [C1]

| Checkpoint             | Refinement and reason                                                                                                                                   |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 Problem framing      | Defined an interval availability forecast for operators because a single present-state reading cannot answer a planning question.                       |
| 2 Tools and memory     | Introduced a ReAct-inspired evidence and feedback cycle. The implementation uses explicit graph steps and persistent records for repeatability.         |
| 3 Retrieval            | Added searchable validated history and guidance. Retrieval changes context and advice while current measurements and calculations remain authoritative. |
| 4 Selective search     | Reserved alternative generation for grounded ambiguity. A small beam and explicit stopping rules limit unnecessary model work.                          |
| 5 Coordination         | Separated generator and critic roles. The MVP adds an optional interpretation role, disabled by default, for a controlled comparison.                   |
| 6 Safety and oversight | Layered input, numerical, runtime, and publication checks. Failed checks produce visible withholding and an opportunity for human review.               |

Implementation choices include a 12-hour horizon, voltage instead of measured SOC, synthetic weather factors, and no battery discharge. LangGraph owns orchestration, MCP owns the bounded source interface, and model nodes interpret evidence. This keeps one functioning system within the capstone's minimal scope.

## Safety reliability and human oversight

A scan of 6,527 MySolArk records found 305.2–394.3 V. The user designated 305.2 V as the fixed battery floor, corresponding to their approximate 30% reserve policy. PAP withholds at or below that reading and never lowers the floor automatically. This is an operating assumption, not a measured SOC conversion or a guarantee of future voltage.

All model roles receive zero tools and schema-checked outputs with evidence citations. Search permits three children per parent, a beam of two, one refinement, and at most eight shared model attempts. Each call has a 45-second limit and no retry. Durable attempt reservations prevent checkpoint recovery from repeating a call. Operators inspect the evidence and request a new validated run when needed. Approval of prose cannot override hard checks. [R1, R2]

## Evaluation methods and results

Deterministic model doubles make required tests reproducible. Cloud adapter checks use actual SDKs with simulated HTTP responses; live cloud performance is unmeasured. Separate browser runs exercise actual MySolArk data and Ollama. These evidence classes do not establish an operational success rate.

| Evaluation              | Observed result                                                               | Interpretation                                          |
|-------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------|
| Local verification      | 44 unit/integration tests and 13 Chromium Playwright checks passed            | Supports tested behavior and UI integration             |
| One real outcome sample | 0.410 kW solar overestimation and 0.076 kW availability error                 | Point-power comparison, not hourly energy accuracy      |
| One actual model pair   | Two roles: 4 calls, 53.035 s, valid. Three roles: 6 calls, 70.223 s, withheld | Calculated kW/kWh stayed identical                      |
| One semantic query      | Reserve guidance ranked first at cosine 0.618                                 | Illustrative retrieval behavior, not retrieval accuracy |

The pair reuses one evidence snapshot, but sequential order and a tiny sample leave warm-up and variability unresolved. It shows added cost without an established quality benefit, so the third role defaults off. PR12 recorded a five-call live run that withheld. PR15 recorded a published sunny-fixture run with four real Ollama calls. Both demonstrate bounded, inspectable outcomes, not advice accuracy. [R3–R8]

## Limitations and next improvements

Synthetic weather, constant demand, unknown live equipment capability, and the absence of a battery model limit forecast usefulness. Source timestamps represent persisted scrapes rather than verified device measurement times. Free-text advice can still be wrong despite valid schemas and citations. Optional LangSmith summary export is tested with a fake client, with no remote export claimed.

Next work should validate a real weather source and equipment limits, collect held-out forecast outcomes across changing conditions, and run repeated, order-balanced agent comparisons. Operator review should evaluate whether explanations help decisions. Physical control remains outside scope.

## Repository and review instructions

Repository: <https://github.com/roboyak/PAP-Agent>. The README explains setup and use. Core code, migrations, tests, the final architecture, and PR evidence support technical review. After installing the documented native prerequisites, run `make bootstrap`, `PAP_PROFILE=macmini-replay make run` for a synthetic source, and `make verify` for the local gate. Real data requires the authorized local source database. [R1]

## Evidence sources

- R1 [Project README and reproducible commands](#)
- R2 [Final MVP architecture and boundaries](#)
- R3 [PR07 observed point-power evaluation](#)
- R4 [PR08 actual local semantic retrieval check](#)
- R5 [PR11 actual two-role and three-role comparison](#)
- R6 [PR12 final verification and live browser evidence](#)
- R7 [PR14 provider selection and simulated SDK checks](#)
- R8 [PR15 inspection and real Ollama walkthrough evidence](#)
- C1 [Earlier report with checkpoint 1–6 source record](#)